MODULE 01 · EXAM STRATEGY - ALL DOMAINS

How a Release Flows Through AWS

One change, followed from a laptop to production and back out again. Every question on this exam is a moment somewhere on that journey.

DOMAIN All six EXAM WEIGHT 100% MODULE 1 of 34

admin@sthithapragna.com

Every DOP question is a moment in the life of **one change**. Find the moment, and the answer narrows itself.

The loop

01
Commit
A change lands in Git and something notices.

02
Build
Compile, package, scan. One immutable artifact.

03
Test
Prove it at the right stage, cheaply first.

04
Deploy
Move traffic to it without an outage.

05
Observe
Metrics, logs, traces and events.

06
Respond
Roll back, remediate, or scale. Automatically.

ON THE EXAM

Respond feeds back into Commit. The exam rewards loops that close themselves without a human.

Where the six domains sit on the loop

DOMAIN	WEIGHT	WHERE IT LIVES
D1 SDLC Automation	22%	Commit, build, test, deploy
D2 Configuration and IaC	17%	Underneath everything: how it all gets created
D3 Resilient Cloud Solutions	15%	Around deploy: surviving load and failure
D4 Monitoring and Logging	15%	Observe
D5 Incident and Event Response	14%	Respond
D6 Security and Compliance	17%	Across every stage, and every account

ON THE EXAM

Domain 1 is the single largest domain, but Domains 2 and 6 together outweigh it. Neither sits at one point on the loop.

The exam's real vocabulary

- **Lambda appears in 43% of questions.** Almost never as the topic. It is the glue between services.
- **EventBridge appears in 29%.** Event, rule, target is the most common shape on the paper.
- CloudWatch 26%, IAM 22%, CloudFormation 20%, Organizations 19%, CodePipeline 18%.
- **One question in five is multiple response.** Choose two, or choose three, with no partial credit.

> ON THE EXAM
>
> We used a bank of several hundred practice questions only to measure emphasis. No question from it appears in this course.

Three shapes that answer most questions

- **Event, rule, target.** Something happens, EventBridge matches it, a target acts: SNS, Lambda, an SSM runbook, a pipeline.
- **The managed feature beats the custom code.** If a service has a built-in switch, the Lambda-and-cron option is the distractor.
- **A role in the target, and a key the target can use.** Every cross-account failure is one or the other.

> ON THE EXAM
>
> Learn to recognise these three shapes and you will pre-sort most options before you have read them fully.

Where the pipeline lives

Log archive
Org CloudTrail, Config
Write once

Git provider
Via CodeConnections
Webhook starts the run

Security tooling
Delegated administrator
GuardDuty, Security Hub

Dev account
Deploy role to assume
First target

Tooling account
CodePipeline and CodeBuild
Artifact bucket and ECR
Customer managed KMS key
Builds happen here, and only here.

Staging account
Deploy role to assume
Tests at scale

Production
Deploy role to assume
Approval or alarms gate

ON THE EXAM

A 'central shared services account' in a question means the centre of this diagram.

Commit: how the loop starts

- Code lives in Git: GitHub, GitLab, Bitbucket, Azure DevOps, or CodeCommit.
- **CodeConnections** links an external provider to AWS. It used to be called CodeStar Connections.
- A **push to main** usually starts the release pipeline. A **pull request** usually starts a test-only build.
- Polling a repository on a timer is the legacy answer. Webhooks and events are the current one.

> ON THE EXAM
>
> CodeConnections was renamed from CodeStar Connections. The CodeStar service itself was discontinued in July 2024.

Build: one artifact, built once

- **CodeBuild** runs the build in a fresh, disposable container, driven by a buildspec file.
- The output is **one immutable artifact**: a zip in S3, an image in ECR, a package in CodeArtifact.
- That same artifact is promoted through dev, staging and production. It is never rebuilt per environment.
- Secrets never sit in the buildspec in plain text. They come from Secrets Manager or Parameter Store.

> ON THE EXAM
>
> Build once, promote many. An option that rebuilds for production has quietly broken the guarantee that you tested what you ship.

Reading a buildspec

```
1  version: 0.2
2  env:
3    secrets-manager:
4      NPM_TOKEN: prod/build/npm:token
5  phases:
6    install:
7      commands: [npm ci]
8    build:
9      commands: [npm test, npm run build]
10 artifacts:
11   files: ['dist/**/*', 'appspec.yml']
```

3-4 The secret is fetched at run time. The file holds only its name.

9 A failing test returns a non-zero exit code, and the build fails.

ON THE EXAM

The build fails because a command exits non-zero. That single rule is how tests gate a pipeline.

Mutable or immutable?

MUTABLE, OR IN-PLACE

Change the servers you have

- Same instances, new code copied on
- Cheap: no extra capacity
- Drift accumulates over time
- Rollback means redeploying the old version

IMMUTABLE

Replace the servers entirely

- New instances or tasks from a new image
- Costs extra capacity while both exist
- Every server is identical to its image
- Rollback means sending traffic back

THE TIEBREAKER

Fastest, safest rollback points to immutable. Lowest cost with no spare capacity points to in-place.

The deployment strategies, in one table

STRATEGY	WHAT HAPPENS	CAPACITY DURING	ROLLBACK
All at once	Every target updated together	Outage possible	Slow redeploy
Rolling	Batches updated in turn	Reduced	Slow redeploy
Rolling, extra batch	New batch first, then roll	Full	Slow redeploy
Immutable	New fleet, then swap	Double, briefly	Fast
Blue/green	New environment, move traffic	Double	Fastest
Canary	Small slice first, then the rest	Double	Fast, little exposure
Linear	Equal steps on a timer	Double	Fast, little exposure

ON THE EXAM

Canary and linear are traffic-shifting flavours of blue/green. Module 07 covers which platforms offer which.

Observe: four kinds of signal

- **Metrics** say something is wrong. CloudWatch, with alarms on thresholds or anomalies.
- **Logs** say what happened. CloudWatch Logs, queried with Logs Insights or Athena.
- **Traces** say where, across services. X-Ray and OpenTelemetry.
- **Events** say something changed. CloudTrail, AWS Health, service events on EventBridge.

> ON THE EXAM
>
> Match the signal to the question: 'which service is slow' wants traces; 'who changed it' wants CloudTrail.

The loop closing itself

01
Deploy
New version takes a slice of traffic.

02
Alarm
5xx errors breach the threshold.

03
Stop
The deployment tool halts the rollout.

04
Revert
Traffic returns to the last good version.

05
Notify
An event tells the team what happened.

ON THE EXAM

Nobody clicked anything. That is the design the exam is looking for.

Respond: when it is not a deployment

- An **EventBridge rule** matches the event and invokes a target.
- Targets that fix things: an **SSM Automation runbook**, a Lambda function, a Step Functions workflow.
- **AWS Config** detects a resource out of policy and can remediate it automatically.
- A person reading an email and running a script is the answer the exam wants you to replace.

> ON THE EXAM
>
> Known problem, known fix: automate it with a runbook. Unknown problem: better signals and a human.

Every station is itself infrastructure

- Pipelines, alarms, rules and accounts are defined as code: CloudFormation, CDK or SAM.
- **StackSets** and **Control Tower** stamp the same baseline into every account and Region.
- Systems Manager keeps the servers themselves in their desired state, and patched.
- Console clicks create drift. The exam treats manual change as the root cause, not the fix.

> ON THE EXAM
>
> If a question describes the same thing being set up by hand in many accounts, the answer is a template deployed at scale.

ALL DOMAINS / ACROSS EVERYTHING

Security at every station

- Each stage runs under its **own role** with only the permissions that stage needs.
- Artifacts crossing accounts are encrypted with a **customer managed KMS key** the targets can use.
- Images and dependencies are **scanned** before they are promoted, not after.
- Guardrails such as **SCPs** and **CloudFormation Hooks** stop bad changes before they exist.

> ON THE EXAM
>
> Preventive beats detective when the question says 'ensure' or 'prevent'. Detective fits 'identify' or 'report'.

Lambda as glue, or Lambda as a trap?

THE RIGHT ANSWER

Connecting what nothing else connects

- A rule target that transforms or decides
- A custom resource, or a lifecycle hook
- Only where no native integration exists
- Short, event-driven, stateless

THE DISTRACTOR

Rebuilding a feature that exists

- A scheduled Lambda polls for a condition
- Custom code does a service's own job
- More to write, test and maintain
- Loses on LEAST operational overhead

THE TIEBREAKER

Before choosing a Lambda option, ask: does the service already have a switch for this? If yes, choose the switch.

The qualifier decides between working answers

WHEN THE QUESTION SAYS	IT IS REALLY ASKING FOR
LEAST operational overhead	The most managed option, even if it costs more
MOST operationally efficient	Fewest moving parts to run and maintain
MOST cost-effective	Cheapest that still meets every stated requirement
LEAST development effort	Configuration over code
Ensure, prevent	A preventive control, applied before the change

ON THE EXAM

On a professional exam several options usually work. The qualifier is what separates them. Module 29 goes deeper.

A warning before we start: the catalogue has moved

IN THE EXAM GUIDE	WHAT HAPPENED
AWS OpsWorks	End of life, May 2024. Systems Manager replaces it
AWS CodeStar	Discontinued, July 2024
AWS Copilot CLI	End of support, June 2026
AWS Proton	End of support, 7 October 2026
AWS App Runner	Closed to new customers, April 2026
CodeDeploy for ECS blue/green	Still valid, but ECS now does it natively

THE RULE

Answer from the exam guide, and know the reality. Module 30 covers every gap with dates.

A team deploys a web application to EC2 instances behind an Application Load Balancer, using a pipeline with a CodeDeploy deployment stage. Last week a release with a bug reached every instance.

Customers saw errors for 40 minutes until an engineer noticed a dashboard and manually redeployed the previous revision. The team wants bad releases reversed automatically, with the least operational overhead.

What should the team do?

A. EventBridge rule on the ALB error alarm invokes a Lambda function that redeploys the previous revision

B. CloudWatch alarm on ALB 5xx errors attached to the deployment group, with automatic rollback enabled

C. Manual approval action before the production stage

D. More unit tests in CodeBuild, so the bug fails the build

◆ PAUSE AND DECIDE

What should the team do?

A EventBridge rule and Lambda redeploy

It would work, but it rebuilds with custom code a rollback CodeDeploy already offers. Loses on least overhead.

B Alarm on the deployment group, automatic rollback

CodeDeploy stops the deployment when the alarm fires and returns to the last good revision. A setting, not a system.

C Manual approval before production

A human cannot see a runtime bug by approving a stage. And the requirement was automatic.

D More unit tests

Worth doing, but it moves the problem to the build station. It reverses nothing once a bug does escape.

THE SIGNAL

Place it on the loop first: this is respond, fed by observe. Then pick the native switch over the Lambda.

A company runs its pipeline in a central tooling account. Artifacts go to an S3 bucket encrypted with the default AWS managed key for S3.

A new deploy stage assumes a role in the production account. The build succeeds, but the production deployment fails with an access denied error when it reads the artifact. The bucket policy already grants the production role s3:GetObject.

What will fix the deployment?

A Move the pipeline into the production account so that no cross-account access is needed

B Disable default encryption on the artifact bucket

C Encrypt artifacts with a customer managed KMS key whose key policy allows the production role to decrypt

D Add s3:GetObject for the production role to the pipeline's own service role

◆ PAUSE AND DECIDE

What will fix the deployment?

A **Move the pipeline into production**

Abandons the central tooling account and puts build rights inside production. It fixes the error by breaking the design.

B **Disable default encryption**

Amazon S3 encrypts every new object regardless. And weakening protection is never the intended fix.

C **Customer managed KMS key the role can use**

The bucket already allowed the read. The key did not. An AWS managed key's policy cannot be edited to admit another account.

D **Permission on the pipeline's own role**

The wrong principal. The production role is doing the read, and it already has S3 permission.

THE SIGNAL

S3 permission present, still access denied, bucket encrypted: the key is the problem. Remember shape three.

Where the course goes from here

- **Modules 2 to 8:** the loop's first half. Pipelines, builds, tests, artifacts, deployment.
- **Modules 9 to 13:** underneath. CloudFormation, CDK, accounts, fleets.
- **Modules 14 to 23:** surviving, watching and responding.
- **Modules 24 to 27:** security across it all. Then confusion pairs, exam technique and four practice papers.

> ON THE EXAM
>
> Every scenario from here on: place it on the loop, spot the shape, then read the qualifier.